

Task 1

1.5

A single `createConnector(string kind)` function with an if/else chain means the function must know about every connector type that exists. Every time Kudu Freight adds a new data source or connector, a developer must modify the existing `createConnector()` function to add another if or else if case. This violates the **Open-Closed Principle**, which states that software should be open for extension but closed for modification, because the existing function must be changed whenever a new connector is introduced.

The Factory Method Pattern follows the Open-Closed Principle by separating object creation into different factory classes. The abstract `ConnectorFactory` declares the factory method `createConnector()` without specifying which concrete connector will be created. Each concrete factory (`CsvFactory`, `PostgresFactory`, `RestApiFactory`) overrides `createConnector()` to return its corresponding concrete connector. **If a new connector is required, a new concrete connector and a matching concrete factory can simply be added without modifying any existing classes.** The system is then easier to maintain, extend, and test. It also reduces coupling because the client depends only on the abstract `ConnectorFactory` and `Connector` interfaces rather than concrete connector classes.

1.6

	<i>Participant role</i>	<i>Reason</i>
Connector	Product	It is the abstract class that defines the interface (<code>extract()</code>) for all connector objects.
CsvConnector	ConcreteProduct	It implements the <code>Connector</code> interface and provides the Csv-specific implementation of <code>extract()</code> .
ConnectorFactory	Creator	It declares the factory method <code>createConnector()</code> that returns a <code>Connector</code> .
CsvFactory	ConcreteCreator	Overrides <code>createConnector()</code> (the factory method) to return a new <code>CsvConnector</code> (<code>ConcreteProduct</code>).

Task 2

2.5

A shallow copy makes a copy of an object's references instead of creating a new object. A change in one object will affect the other. Whereas a deep copy creates a new object, with its own copy of the data. The deep copy is independent of the copied object, so any change in one will not affect the other. The clone() function must produce a deep copy of the prototype so that all the clones are independent and modifications on one of the clones does not impact the other clones.

2.6

The registry returns clone(s) rather than the stored pointer because if they shared the same prototype object, if one pipeline modified the transformation's state, the other pipeline would be affected. This could lead to issues such as dangling pointers and double deletion. Returning clones gives both pipelines their own separate copy to avoid these issues.

Task 3

3.5

run() is deliberately not virtual because it specifies the specific order in which connect(), extract(), transform() and load() should be executed in each subclass. If the subclasses could override it, this could lead to some of the functions not being executed at all or in the wrong order. Making run() non-virtual guarantees that every pipeline follows the same processing sequence.

3.6

run()- template method,

connect()- concrete step

extract() – primitive operation

transform() – concrete step

load() – primitive operation

The Factory method created the connector object without the pipeline needing to know which concrete connector class is being used. The template method makes sure that all the pipeline objects follow the same logical sequence/algorithm. They do not overlap because they do not perform the same function. The template method manages the algorithm while the factory class manages object creation.

Task 4

4.4

Wide Interface: exists between originator(pipeline) and memento(RunCheckpoint)

The wide interface contains everything and exposes the full captured state. The wide interface allows reading, writing and restoring the state, hence only the **Originator** can use it.

Pipeline is the only class that calls *getStage()* and *getRecords()* - hence it is the only one that knows what that state contains and how to use it(via *creatCheckpoint()* and *restore()*). Pipeline- the originator- uses it.

Narrow Interface: exists between memento(RunCheckpoint) and caretaker(CheckpointManager)

The narrow interface does not expose the internal state hence it is used by the **Caretaker(CheckpointManager)**. The caretaker only needs to store the memento, return it later on and delete if necessary.

4.5

1. During a normal run, after transform() finishes the pipeline will call **createCheckpoint()**
2. createCheckpoint() then creates a RunCheckpoint (Memento) containing the Pipeline's current stage and records.
3. The Pipeline passes the newly created RunCheckpoint to the CheckpointManager (Caretaker).
4. The CheckpointManager stores the checkpoint by calling **save(checkpoint)**, which appends it to its list of saved checkpoints.
5. load() runs next and the program crashes at 02:00 before it completes.
6. Upon restart a new pipeline object will be constructed with the same type of factory as the failed run.
7. To recover, the program asks the CheckpointManager for the most recent checkpoint by calling **undo()**.
8. **undo()** is called, popping and returning the most recently saved RunCheckpoint.
9. The returned checkpoint is passed back to the Pipeline.
10. The new pipeline calls **restore(checkpoint)**, which sets its stage and records back to exactly where the previous successful checkpoint left off (after transform() and before load())
11. The checkpoint is deleted once it has passed out of undo(), and the run resumes from load() instead of starting the entire run again(rerunning connect(), extract(), and transform() from scratch).

4.6

Class	Participant roles	Reason
Pipeline	Originator	It creates checkpoints of its own state uses <i>createCheckpoint()</i> and restores its state using <i>restore()</i> .
RunCheckpoint	Memento	It stores the Pipeline's saved state (stage and records)
CheckpointManager	Caretaker	It stores, manages and returns RunCheckpoint (the memento) objects without examining or modifying the contents.